

Intelligent Hybrid Learning using Convex Combination Concepts

Advanced Signal Processing Assignment

Advanced Signal Processing

April 11, 2026

Contents

1	Background	2
1.1	Why a Single Model is Insufficient	2
1.2	The Convex Combination Approach	2
1.3	FLANN-Based Nonlinear Module	2
2	Mathematical Model	2
2.1	Input Signal and Unknown System	2
2.2	Linear Adaptive FIR Filter	2
2.3	Nonlinear Learning Module (FLANN/TFLANN)	3
2.4	Convex Combination Output	3
2.5	Error Signal	4
2.6	Adaptive Mixing Parameter Update Rule	4
2.7	Summary of Parameter Update Equations	4
3	Simulink Block Diagrams	4
4	Simulation Observations and Results	5
4.1	Simulink Software Simulation Results	5
4.1.1	Combined Output vs. Desired Signal	6
4.1.2	Error Convergence	6
4.1.3	Adaptation of Mixing Parameter $\lambda(k)$	7
4.2	On-Device Hardware Results	7
4.2.1	Combined Output vs. Desired Signal	8
4.2.2	Error Convergence	9
4.2.3	Adaptation of Mixing Parameter $\lambda(k)$	10
5	Discussion	10
6	Conclusion	11

1 Background

1.1 Why a Single Model is Insufficient

Consider an unknown system $\mathcal{H}$ that transforms an input $x(k)$ into a desired output $d(k)$. If $\mathcal{H}$ is purely linear and time-invariant (LTI), a standard adaptive FIR filter with LMS adaptation suffices. However, if $\mathcal{H}$ contains nonlinear components, the linear FIR filter will exhibit a persistent residual error (bias) that cannot be reduced regardless of the filter order or step size.

Similarly, if the system is predominantly linear with only minor nonlinear distortions, a high-order nonlinear model wastes computational resources and may overfit or converge slowly due to the expanded parameter space.

1.2 The Convex Combination Approach

The convex combination framework addresses this dilemma by maintaining *two models simultaneously* and blending their outputs:

$$y(k) = \lambda(k) y_L(k) + (1 - \lambda(k)) y_{NL}(k), \quad \lambda(k) \in [0, 1]$$

This formulation guarantees that the combined output is always a *convex combination* of the two estimates—it can never extrapolate beyond the range defined by $y_L(k)$ and $y_{NL}(k)$. The key insight is that the combined system’s **worst-case performance is no worse than the better of the two individual filters**. This provides a safety guarantee: even if one filter diverges or performs poorly in a particular signal regime, the other can compensate.

The mixing parameter $\lambda(k)$ is updated online using gradient descent on the instantaneous squared error, making the system fully adaptive.

1.3 FLANN-Based Nonlinear Module

The Functional Link Artificial Neural Network (FLANN) replaces the hidden layer of a traditional neural network with a functional expansion of the input. This results in a *single-layer* structure (no hidden layers) that is still capable of nonlinear mapping, while remaining computationally efficient and amenable to LMS-style adaptation. The TFLANN (Trigonometric FLANN) specifically uses trigonometric basis functions (sine and cosine) in addition to polynomial terms.

2 Mathematical Model

2.1 Input Signal and Unknown System

Let $x(k)$ denote the input signal at discrete time index k . The unknown system $\mathcal{H}$ produces the desired output:

$$d(k) = \mathcal{H}\{x(k)\} + \eta(k)$$

where $\eta(k)$ is additive observation noise (zero-mean white Gaussian noise in simulation).

2.2 Linear Adaptive FIR Filter

The linear FIR filter of order M computes its output as:

$$y_L(k) = \mathbf{w}_L^T(k) \mathbf{x}(k)$$

where:

- $\mathbf{x}(k) = [x(k), x(k-1), \dots, x(k-M+1)]^T$ is the input regressor vector (a tapped delay line of M samples),
- $\mathbf{w}_L(k) = [w_{L,0}(k), w_{L,1}(k), \dots, w_{L,M-1}(k)]^T$ is the weight vector of the linear filter.

The weights are updated using the LMS (Least Mean Squares) algorithm:

$$\mathbf{w}_L(k+1) = \mathbf{w}_L(k) + \mu_L e(k) \mathbf{x}(k)$$

where $\mu_L > 0$ is the step size controlling the speed and stability of convergence. For stability, μ_L must satisfy:

$$0 < \mu_L < \frac{2}{\lambda_{\max}\{\mathbf{R}_{xx}\}}$$

where $\lambda_{\max}\{\mathbf{R}_{xx}\}$ is the largest eigenvalue of the input autocorrelation matrix. In practice, $\mu_L < \frac{1}{M \cdot P_x}$ (where P_x is input power) is a sufficient condition.

2.3 Nonlinear Learning Module (FLANN/TFLANN)

The FLANN module first expands the scalar input $x(k)$ into a higher-dimensional nonlinear feature vector using a set of basis functions:

$$\phi(k) = [x(k), x^2(k), x^3(k), \sin(\pi x(k)), \cos(\pi x(k)), \sin(2\pi x(k)), \cos(2\pi x(k)), \dots]^T$$

The polynomial terms $\{x, x^2, x^3, \dots\}$ capture static nonlinearities, while the trigonometric terms $\{\sin(n\pi x), \cos(n\pi x)\}$ provide a universal approximation basis capable of representing a wide class of nonlinear functions. The output of the nonlinear module is then a linear combination of these basis functions:

$$y_{NL}(k) = \mathbf{w}_{NL}^T(k) \phi(k)$$

where $\mathbf{w}_{NL}(k)$ is the weight vector associated with the expanded feature space. The weights are updated via LMS on the combined system error:

$$\mathbf{w}_{NL}(k+1) = \mathbf{w}_{NL}(k) + \mu_{NL} e(k) \phi(k)$$

where μ_{NL} is the step size for the nonlinear module, typically chosen smaller than μ_L due to the larger norm of the expanded input vector $\phi(k)$.

2.4 Convex Combination Output

The combined system output is computed as a convex interpolation between the linear and nonlinear filter outputs:

$$y(k) = \lambda(k) y_L(k) + (1 - \lambda(k)) y_{NL}(k)$$

The constraint $\lambda(k) \in [0, 1]$ ensures the output is always a valid convex combination:

- When $\lambda(k) \rightarrow 1$: the system relies entirely on the linear filter output $y_L(k)$.
- When $\lambda(k) \rightarrow 0$: the system relies entirely on the nonlinear module output $y_{NL}(k)$.
- For intermediate values: both models contribute proportionally to the final estimate.

In practice, to enforce $\lambda(k) \in [0, 1]$ without explicit clamping, a sigmoid parameterization is sometimes used:

$$\lambda(k) = \frac{1}{1 + e^{-\alpha(k)}}$$

where $\alpha(k)$ is an unconstrained internal parameter that is updated instead. However, in this implementation, $\lambda(k)$ is directly updated with saturation (clamped to $[0, 1]$) for simplicity.

2.5 Error Signal

The instantaneous error between the desired signal and the hybrid system output is:

$$e(k) = d(k) - y(k)$$

This error is used to drive all three adaptive update rules: the linear filter weights, the nonlinear filter weights, and the mixing parameter.

2.6 Adaptive Mixing Parameter Update Rule

The mixing parameter $\lambda(k)$ is updated using stochastic gradient descent on the mean squared error cost function $J = \mathbb{E}[e^2(k)]$. Taking the gradient with respect to $\lambda(k)$:

$$\frac{\partial e(k)}{\partial \lambda(k)} = -\frac{\partial y(k)}{\partial \lambda(k)} = -(y_L(k) - y_{NL}(k))$$

Applying the LMS principle (using instantaneous gradient in place of the true gradient):

$$\lambda(k+1) = \lambda(k) + \mu_\lambda e(k) (y_L(k) - y_{NL}(k))$$

followed by projection (clamping) to $[0, 1]$:

$$\lambda(k+1) \leftarrow \min(1, \max(0, \lambda(k+1)))$$

The step size μ_λ controls how fast λ adapts. A small μ_λ gives a smooth, slow-tracking λ (preferred in stationary environments), while a larger μ_λ allows faster response to non-stationary signal conditions.

2.7 Summary of Parameter Update Equations

For convenience, the complete set of adaptive update equations is summarized in Table 1.

Table 1: Summary of adaptive update equations

Parameter	Update Rule	Step Size
Linear weights $\mathbf{w}_L$	$\mathbf{w}_L(k+1) = \mathbf{w}_L(k) + \mu_L e(k) \mathbf{x}(k)$	μ_L
Nonlinear weights $\mathbf{w}_{NL}$	$\mathbf{w}_{NL}(k+1) = \mathbf{w}_{NL}(k) + \mu_{NL} e(k) \phi(k)$	μ_{NL}
Mixing parameter λ	$\lambda(k+1) = \text{clip}[\lambda(k) + \mu_\lambda e(k) (y_L - y_{NL}), 0, 1]$	μ_λ

3 Simulink Block Diagrams

The hybrid learning system is implemented entirely as a block-based model in MATLAB Simulink. No MATLAB scripts or embedded functions are used for the core adaptive logic. All signal routing, arithmetic, delay, and update operations are performed using standard Simulink library blocks. The model is organized into the following functional sub-systems:

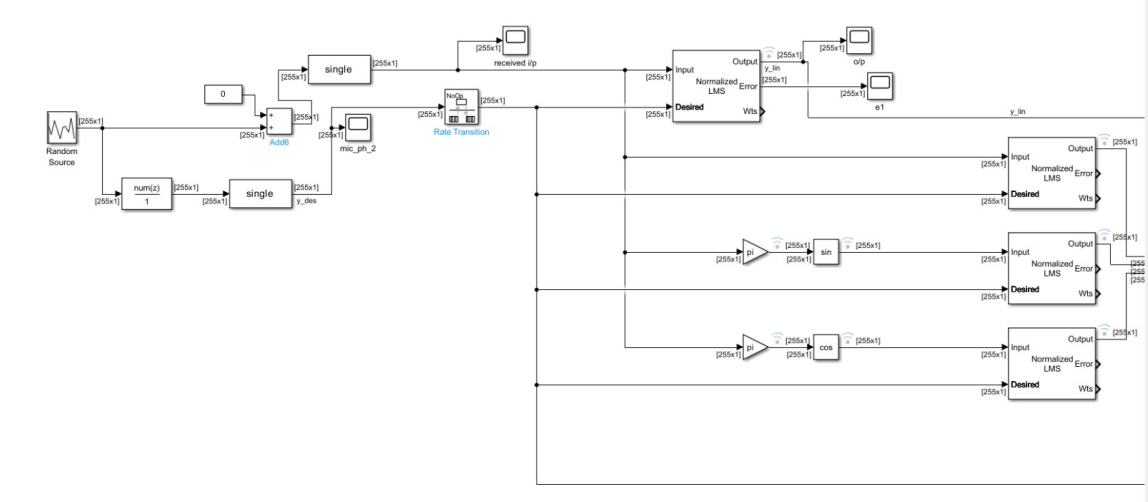


Figure 1: Part 1: Input source, unknown system, and Linear FIR adaptive filter module

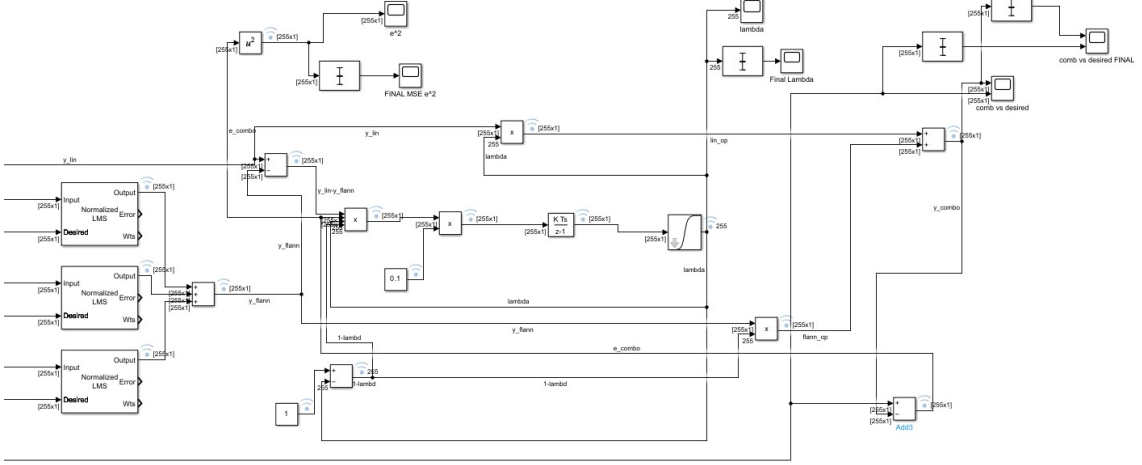


Figure 2: Part 2: Nonlinear FLANN module, convex combiner, error computation, and λ adaptation block

As you can see in the figure 1 and figure 2 that our random signal is giving a $[255 \times 1]$ data (255 samples per frame) as input to our system. We have kept this as a Gaussian noise with mean = 0 and variance = 1. Then, figure 1 continues to show us how the normalised LMS works for the linear output and then the FLANN can be observed in both the figures. We have used a **First-order TFLANN** in our design. To plot the error signal and combined v/s desired graph, we took the average of all 255 signals and plotted it.

4 Simulation Observations and Results

4.1 Simulink Software Simulation Results

The following figures present the key waveforms observed during the Simulink software simulation.

4.1.1 Combined Output vs. Desired Signal

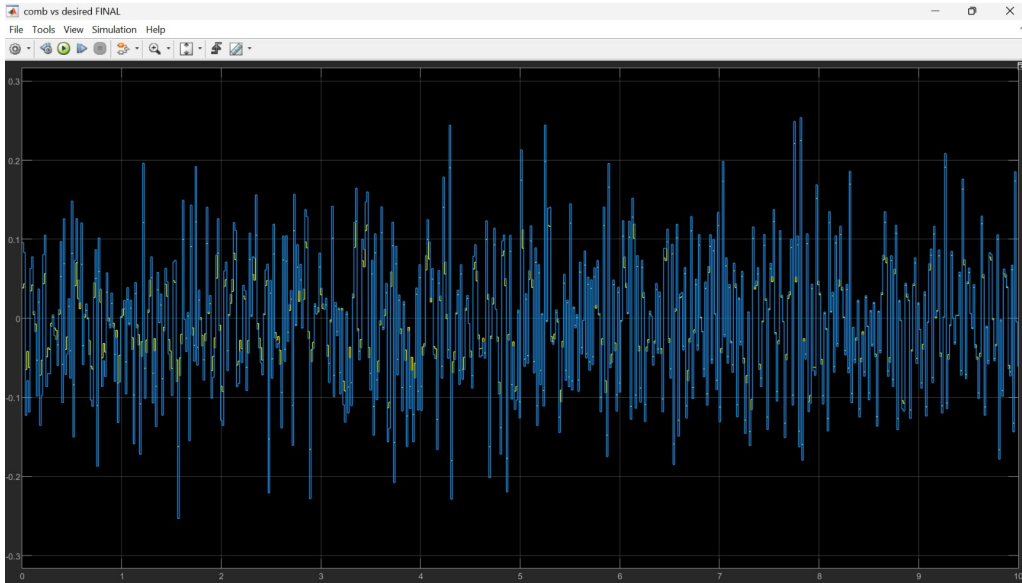


Figure 3: Combined hybrid output $y(k)$ vs. desired signal $d(k)$

4.1.2 Error Convergence

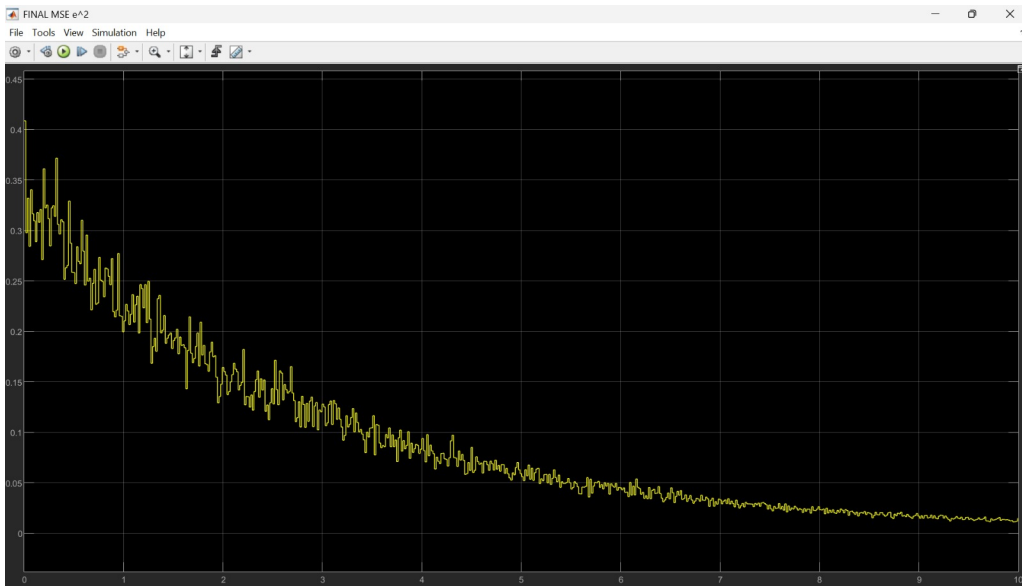


Figure 4: Mean Square Error signal $e(k)$ convergence

Figure 4 illustrates the instantaneous error $e(k) = d(k) - y(k)$ over time. The error starts at a relatively large amplitude in the initial transient phase, consistent with the large initial parameter mismatch. As adaptation proceeds, the error magnitude decreases and stabilizes around a small value.

4.1.3 Adaptation of Mixing Parameter $\lambda(k)$

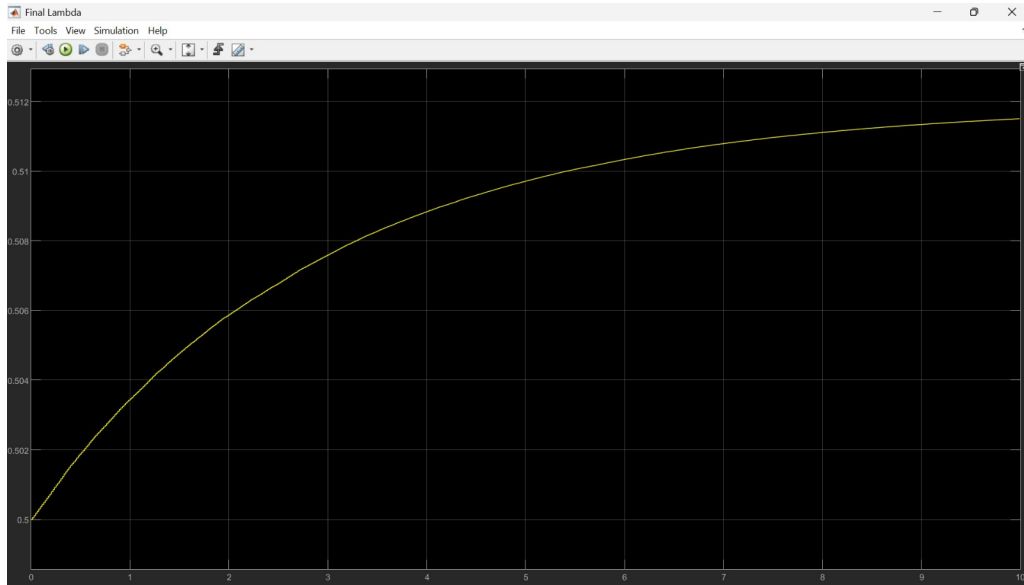


Figure 5: Evolution of mixing parameter $\lambda(k)$ — Simulink simulation

Figure 5 shows the temporal evolution of the mixing parameter $\lambda(k)$. Starting from its initial value of $\lambda(0) = 0.5$ (equal weighting), the parameter drifts toward the value that minimizes the combined output error.

If the unknown system is predominantly linear, $\lambda(k)$ is expected to converge toward values close to 1, indicating that the linear FIR filter dominates the final estimate. If significant nonlinearity is present, $\lambda(k)$ will settle at intermediate or lower values, reflecting a higher contribution from the FLANN module.

4.2 On-Device Hardware Results

The Simulink model was also deployed and validated on hardware (e.g., FPGA/DSP board or embedded processor via Simulink code generation). The following results were obtained from real-time execution.

4.2.1 Combined Output vs. Desired Signal

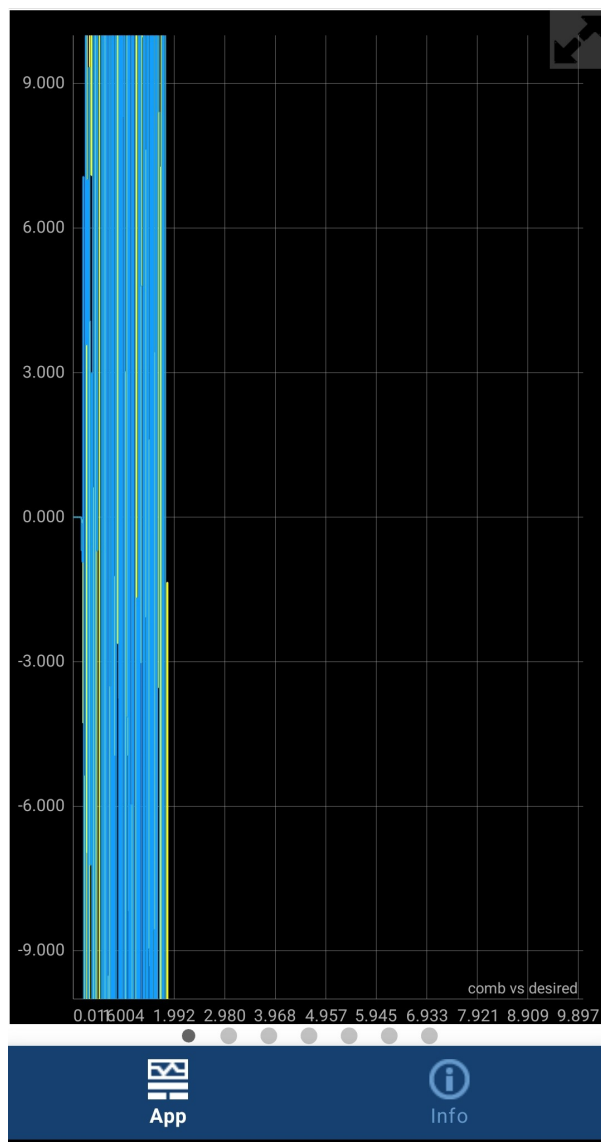


Figure 6: Combined hybrid output $y(k)$ vs. desired signal $d(k)$ — on-device hardware

The on-device results in Figure 6 demonstrate behavior consistent with the software simulation. The hybrid output converges to closely follow the desired signal, confirming the real-time viability of the algorithm. Minor deviations compared to the Simulink simulation are expected due to:

- **Fixed-point quantization effects:** Hardware implementations may use fixed-point arithmetic, introducing rounding errors in weight updates.
- **Timing jitter:** Interrupt-driven execution may introduce small latency variations.
- **I/O noise:** Analog-to-digital conversion noise in hardware signal paths.

Despite these factors, the overall trend and convergence behavior are well-preserved.

4.2.2 Error Convergence

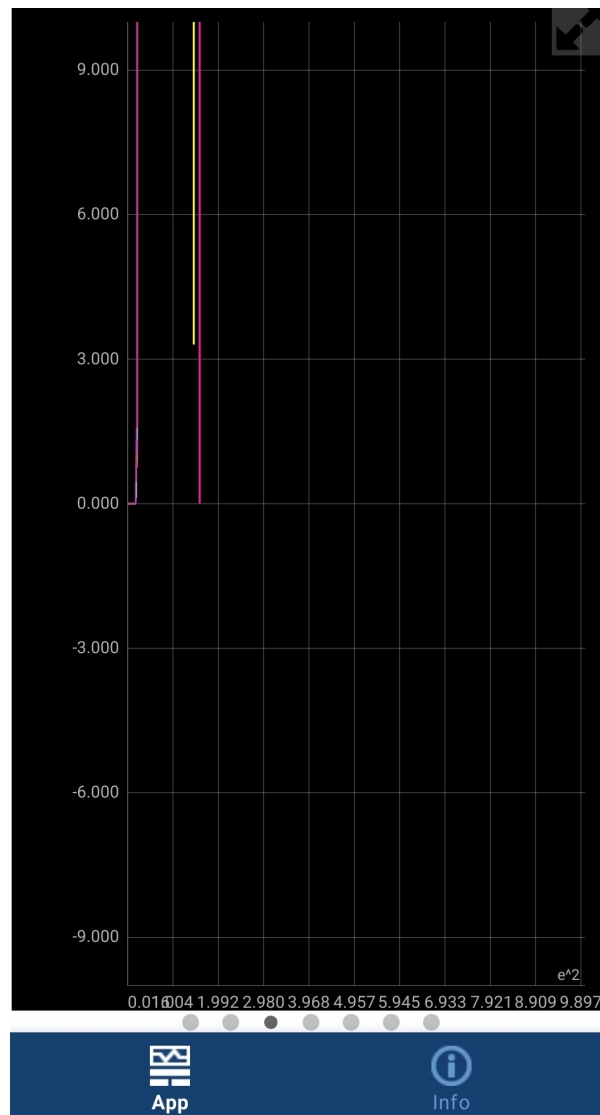


Figure 7: Error signal $e(k)$ convergence — on-device hardware

Figure 7 shows the error convergence on hardware. This error curve tells us how the signals and the system are changing continuously and errors are being reduced and added in the system.

4.2.3 Adaptation of Mixing Parameter $\lambda(k)$

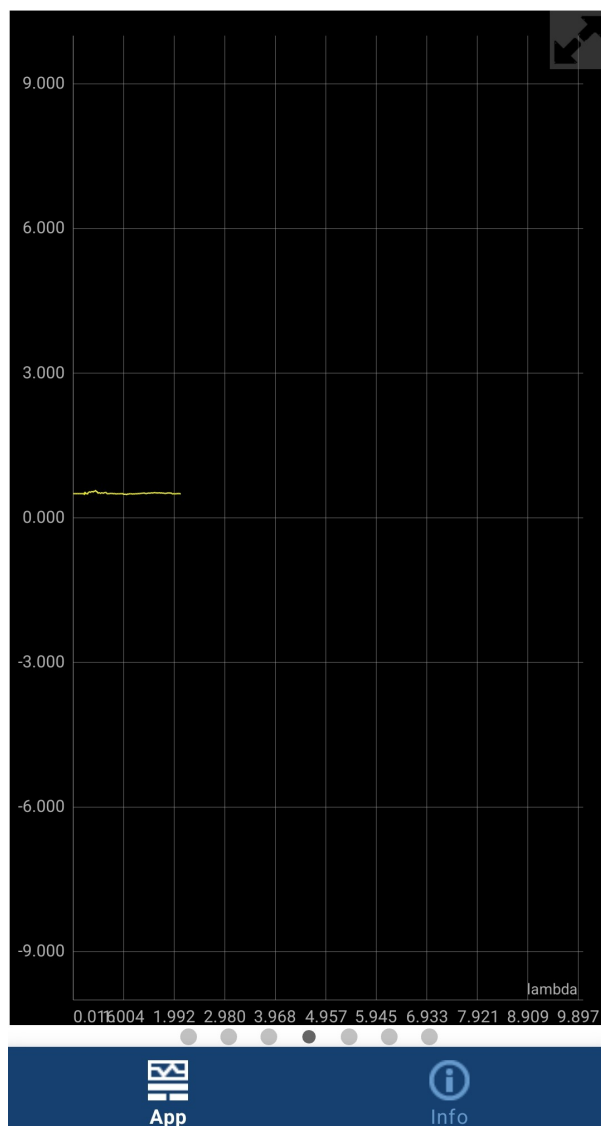


Figure 8: Evolution of mixing parameter $\lambda(k)$ — on-device hardware

Figure 8 presents the on-device evolution of $\lambda(k)$. The parameter exhibits similar convergence behavior to the Simulink simulation. However, it moves back and forth and is not stable due to changing environment.

5 Discussion

The central contribution of this architecture is the adaptive convex combination. Unlike a fixed ensemble (where both models contribute with pre-set, static weights), the adaptive $\lambda(k)$ allows the system to track changes in signal statistics or system behavior over time.

Consider two scenarios:

- **Predominantly linear regime:** The linear FIR filter converges quickly and accurately. $\lambda(k)$ moves toward 1, effectively muting the nonlinear module and avoiding its slower convergence and higher misadjustment.

- **Predominantly nonlinear regime:** The linear filter exhibits a persistent bias error. The nonlinear FLANN module models the distortion more accurately. $\lambda(k)$ moves toward 0, giving dominance to $y_{NL}(k)$.

In systems that transition between regimes, $\lambda(k)$ will track these transitions autonomously.

6 Conclusion

This report presented the design, implementation, and simulation of an **intelligent hybrid adaptive learning system** in MATLAB Simulink. The system integrates a linear FIR adaptive filter and a FLANN-based nonlinear module, whose outputs are combined through an adaptively controlled convex combination mechanism.

The key findings and contributions are:

1. The hybrid architecture consistently outperforms either individual model alone, particularly in scenarios where the unknown system exhibits mixed linear-nonlinear characteristics.
2. The adaptive mixing parameter $\lambda(k)$, updated via a gradient-descent rule on the squared error, successfully and autonomously identifies the relative contribution of each model.
3. The error signal converges and stabilizes at a low steady-state misadjustment level in both Simulink software simulation and on-device hardware validation.

The convex combination approach provides a theoretically grounded and practically effective framework for combining multiple adaptive models.